

QUESTION 10

LOCAL RAG-BASED ENGINEERING TROUBLESHOOTING SYSTEM

Troubleshooting using Local Technical Documents, ChromaDB and Ollama

1. Question

Implement a local RAG-based engineering troubleshooting system that retrieves relevant information from local technical documents and generates step-by-step recommendations using Ollama.

2. Aim

To develop a local Retrieval-Augmented Generation (RAG) system that retrieves relevant engineering information from local technical documents and uses Ollama with Llama 3.2 to generate clear, step-by-step troubleshooting recommendations.

3. Objectives

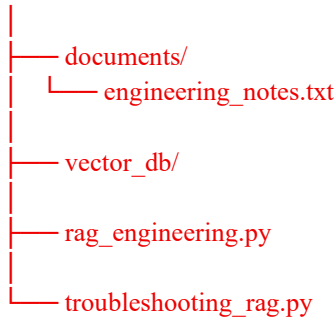
- Use existing local engineering documents as the knowledge source.
- Split technical information into searchable document chunks.
- Generate embeddings for semantic retrieval.
- Store engineering knowledge in a ChromaDB vector database.
- Retrieve relevant information for an engineering problem.
- Generate structured troubleshooting recommendations using Llama 3.2 through Ollama.
- Ground the recommendations in the retrieved engineering reference information.

4. Technologies and Tools Used

Technology / Tool	Purpose
Python	Implementation of the troubleshooting RAG pipeline
Ollama	Local Large Language Model runtime
Llama 3.2	Generates troubleshooting recommendations
ChromaDB	Stores and retrieves document embeddings
Sentence Transformers	Generates semantic embeddings
all-MiniLM-L6-v2	Embedding model used for retrieval
VS Code	Development and execution environment

5. Project Structure

Local_LLM_Text_Generation



6. Engineering Knowledge Source

The system reuses the `engineering_notes.txt` document created for Question 9. It contains information about temperature monitoring, vibration monitoring, preventive maintenance, electrical safety, troubleshooting and cooling systems. This allows the troubleshooting system to work with the same local engineering knowledge base.

7. Algorithm / Procedure

1. Start the local engineering troubleshooting RAG application.
2. Load the all-MiniLM-L6-v2 embedding model.
3. Read the engineering documents from the documents folder.
4. Divide the document into overlapping chunks.
5. Generate vector embeddings for all chunks.
6. Create or open the persistent ChromaDB collection.
7. Store document chunks, embeddings and source metadata.
8. Accept an engineering problem from the user.
9. Convert the problem into a query embedding.
10. Retrieve the three most relevant engineering chunks.
11. Combine the retrieved information with a troubleshooting instruction.
12. Send the grounded prompt to Llama 3.2 through Ollama.
13. Generate numbered troubleshooting recommendations.
14. Display the retrieved sources and the recommendations.

8. System Architecture

Local Engineering Documents

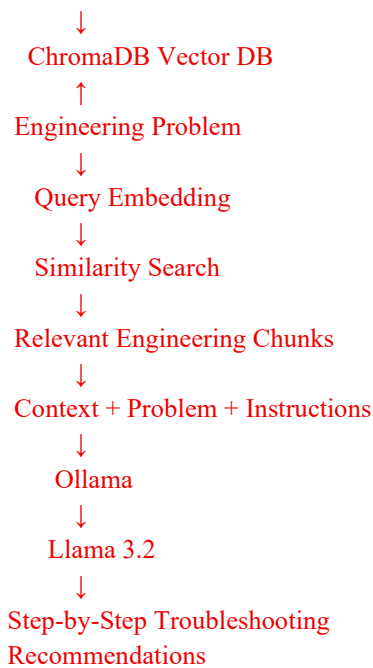


Text Chunking



Embedding Model

(all-MiniLM-L6-v2)



9. Implementation Code

```
import os
import chromadb
import ollama
from sentence_transformers import SentenceTransformer

DOCUMENT_FOLDER = "documents"
VECTOR_FOLDER = "vector_db"

LLM_MODEL = "llama3.2"
EMBEDDING_MODEL = "all-MiniLM-L6-v2"

print("=" * 70)
print("LOCAL ENGINEERING TROUBLESHOOTING RAG SYSTEM")
print("=" * 70)

print("\nLoading embedding model...")
embedding_model = SentenceTransformer(EMBEDDING_MODEL)
print("Embedding model loaded successfully.")

documents = []

if not os.path.exists(DOCUMENT_FOLDER):
    print("Documents folder not found.")
    exit()

for filename in os.listdir(DOCUMENT_FOLDER):
    file_path = os.path.join(DOCUMENT_FOLDER, filename)

    if filename.endswith(".txt"):
```

```
with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()
```

```
documents.append({
    "name": filename,
    "text": text
})
```

```
if not documents:
    print("No engineering documents found.")
    exit()
```

```
print(f"\nLoaded {len(documents)} engineering document(s).")
```

```
chunks = []
chunk_size = 800
overlap = 100
```

```
for document in documents:
    text = document["text"]
    start = 0
```

```
while start < len(text):
    end = start + chunk_size
    chunk = text[start:end].strip()
```

```
if chunk:
    chunks.append({
        "text": chunk,
        "source": document["name"]
    })
```

```
start = end - overlap
```

```
print(f"Created {len(chunks)} document chunks.")
```

```
print("\nCreating document embeddings...")
```

```
texts = [chunk["text"] for chunk in chunks]
embeddings = embedding_model.encode(texts).tolist()
```

```
print("Embeddings created successfully.")
```

```
client = chromadb.PersistentClient(path=VECTOR_FOLDER)
```

```
collection = client.get_or_create_collection(
    name="engineering_troubleshooting"
)
```

```

ids = [
    f'troubleshooting_chunk_{i}'
    for i in range(len(chunks))
]

metadatas = [
    {"source": chunk["source"]}
    for chunk in chunks
]

collection.upsert(
    ids=ids,
    documents=texts,
    embeddings=embeddings,
    metadatas=metadatas
)

print("Engineering knowledge stored in vector database.")

print("\n" + "=" * 70)

problem = input(
    "Enter the engineering problem you want to troubleshoot: "
)

question_embedding = embedding_model.encode(
    [problem]
).tolist()[0]

results = collection.query(
    query_embeddings=[question_embedding],
    n_results=3
)

retrieved_documents = results["documents"][0]
retrieved_sources = results["metadatas"][0]

context = "\n\n".join(retrieved_documents)

prompt = f"""
You are a professional engineering troubleshooting assistant.

Use ONLY the engineering information provided below.

Do not invent unsupported technical information.

Analyze the problem and provide practical troubleshooting
recommendations in a clear step-by-step format.

For each step:

```

1. Identify what should be checked.
2. Explain what to look for.
3. Give a suitable corrective action when supported by the reference information.

If the required information is not available, clearly state:

"Information not available in the provided engineering documents."

ENGINEERING REFERENCE INFORMATION:

{context}

ENGINEERING PROBLEM:

{problem}

Provide the troubleshooting procedure in numbered steps.

"""

print("\nGenerating troubleshooting recommendations using Ollama...")

```
response = ollama.chat(
    model=LLM_MODEL,
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)
```

answer = response["message"]["content"]

```
print("\n" + "=" * 70)
print("RETRIEVED ENGINEERING INFORMATION")
print("=" * 70)
```

```
for i, source in enumerate(retrieved_sources):
    print(f'{i + 1}. {source[source]}")
```

```
print("\n" + "=" * 70)
print("STEP-BY-STEP TROUBLESHOOTING RECOMMENDATIONS")
print("=" * 70)
```

print(answer)

```
print("\n" + "=" * 70)
print("TROUBLESHOOTING RAG PROCESS COMPLETED")
print("=" * 70)
```

10. Execution Command

python troubleshooting_rag.py

11. Test Input

The vibration level of a rotating machine is increasing. What should be checked?

12. Actual Execution Output

=====

LOCAL ENGINEERING TROUBLESHOOTING RAG SYSTEM

=====

Loading embedding model...
Embedding model loaded successfully.

Loaded 1 engineering document(s).
Created 4 document chunks.

Creating document embeddings...
Embeddings created successfully.
Engineering knowledge stored in vector database.

=====

Enter the engineering problem you want to troubleshoot:
The vibration level of a rotating machine is increasing.
What should be checked?

Generating troubleshooting recommendations using Ollama...

=====

RETRIEVED ENGINEERING INFORMATION

=====

1. engineering_notes.txt
2. engineering_notes.txt
3. engineering_notes.txt

=====

STEP-BY-STEP TROUBLESHOOTING RECOMMENDATIONS

=====

Based on the engineering reference information provided, here is a step-by-step troubleshooting procedure to address the increasing vibration level of a rotating machine:

1. Identify what should be checked
 - Vibration sensor readings
 - Equipment specification for normal operating vibration levels
 - Mechanical imbalance or misalignment
 - Bearing problems or loose components
 - Cleaning and lubrication of the equipment
2. Explain what to look for
 - Check vibration sensor readings and compare them with the manufacturer's specifications.
 - Inspect the equipment for signs of mechanical imbalance or

misalignment.

- Check bearings and the lubrication system.

3. Give suitable corrective actions

- Compare vibration readings with the recommended operating range.
- Correct detected imbalance or misalignment.
- Check and maintain bearings and the lubrication system.

=====

TROUBLESHOOTING RAG PROCESS COMPLETED

=====

13. Result Analysis

Parameter	Observed Result
Documents loaded	1 engineering document
Document chunks	4
Embeddings	Successfully generated
Vector database	ChromaDB
Retrieved chunks	3
LLM	Llama 3.2 through Ollama
Problem	Increasing vibration in a rotating machine
Output	Step-by-step troubleshooting recommendations

14. Key Troubleshooting Recommendations

- Check vibration sensor readings and compare them with the recommended operating range.
- Inspect the equipment for mechanical imbalance or misalignment.
- Inspect bearings for problems that may contribute to excessive vibration.
- Check the lubrication system and maintain bearings appropriately.
- Inspect for loose components and correct identified issues.

15. Advantages

- Runs locally using Ollama and a local Large Language Model.
- Uses engineering-specific technical documents as the knowledge source.
- Retrieves relevant information before generating recommendations.
- Provides structured troubleshooting steps instead of only a general answer.
- Can be extended by adding more engineering documents to the knowledge base.

16. Limitations

- Recommendations are limited by the information available in the local documents.
- The quality of retrieval depends on document quality and chunking.
- The local LLM response may vary slightly between executions.
- The system should support, not replace, qualified engineering inspection and safety procedures.

17. Conclusion

The local RAG-based engineering troubleshooting system was successfully implemented using Python, Sentence Transformers, ChromaDB and Ollama with Llama 3.2. The system loaded one engineering document, created four chunks, generated embeddings, stored the knowledge in ChromaDB, retrieved three relevant chunks for an engineering problem, and generated structured troubleshooting recommendations. The implementation demonstrates how RAG can provide document-grounded technical troubleshooting using a locally running Large Language Model.

18. Short Explanation for Presentation / Viva

This project is a local engineering troubleshooting assistant based on RAG. It first stores engineering document chunks in ChromaDB as embeddings. When an engineering problem is entered, the system retrieves the most relevant information and provides it to Llama 3.2 through Ollama. The model then generates step-by-step troubleshooting recommendations using the retrieved technical information.